



Sharif University of Technology
Department of Electrical Engineering

Report of Course Project

Signals and Systems

Project Phase II

By:

Dorsa Akbari
Melika Pourakbar

Professor:

Dr. B. Khalaj

Head Teacher Assistant:

Mohammad Eskandari

Spring 2025 (1403-2)

Contents

1	Theory: Introduction to License Plate Recognition (LPR)	2
1.1	Two-Dimensional Correlation	2
1.1.1	Task 1	2
1.1.2	Task 2	3
1.2	Image Blurring and Degradation Modeling	3
1.2.1	Task 1	3
1.2.2	Task 2	4
1.3	Sampling Rate and Resolution Effects	4
1.3.1	Task	4
2	Implementation	7
2.1	Ideal Case - Controlled Environment	7
2.1.1	Task 1	7
2.1.2	Task 2	7
2.1.3	Task 3	8
2.2	Realistic Case – Simulated Real-World Conditions	11
2.2.1	Task 1	11
2.2.2	Task 2	11
2.2.3	Task 3	12
2.2.4	Task 4	12
2.2.5	Task 5	12
2.2.6	Task 6	14

1 Theory: Introduction to License Plate Recognition (LPR)

In the real world, license plate recognition (LPR) systems are essential tools used in traffic monitoring, access control, and law enforcement. These systems rely on a series of signal and image processing techniques to extract the characters from license plates and identify them correctly.

The core method we will employ for character recognition is correlation. The correlation process compares segments of the license plate image with a set of reference characters to find the best match.

However, real-world LPR systems face several challenges such as image blurring due to motion or focus, low sampling rates from limited hardware, or noisy imaging conditions. We simulate some of these challenges in this project to understand their effects on signal integrity and recognition accuracy.

1.1 Two-Dimensional Correlation

In the project, we will perform character recognition using correlation. This subsection investigates how the one-dimensional correlation method can be adapted for image processing and two-dimensional tasks.

1.1.1 Task 1

The mathematical definition of correlation between two signals and its standard formula, plus what the correlation value represents:

Correlation is a mathematical technique used to measure the similarity or relationship between two signals. In its most fundamental form, the cross-correlation between two continuous signals $x(t)$ and $y(t)$ is defined mathematically as:

$$r_{xy}(\tau) = \int_{-\infty}^{\infty} x(t) \cdot y(t - \tau) dt$$

where τ is the time lag. For discrete signals $x[n]$ and $y[n]$, the correlation is:

$$r_{xy}[m] = \sum_{n=-\infty}^{\infty} x[n] \cdot y[n - m]$$

where m is the discrete lag parameter.

The correlation value represents the degree of similarity between two signals as a function of their relative displacement. A correlation coefficient close to +1 indicates a strong positive relationship, meaning the signals are very similar. A correlation coefficient close to -1 indicates a strong negative relationship, while a value near 0 suggests little to no linear relationship between the signals. In template matching applications, the peak correlation value indicates the best match position between a template and the target signal.

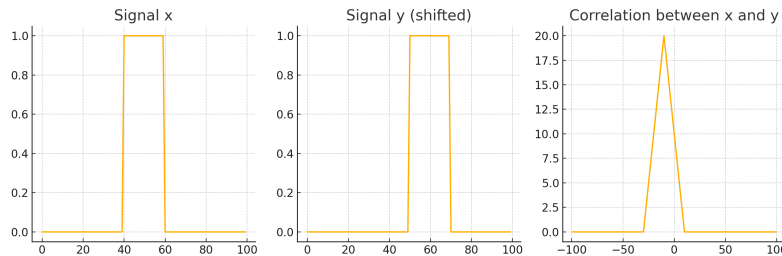


Figure 1: An example of correlation between two 1D signals with a time shift

Sources used for this task: [\[1\]](#) [\[2\]](#) [\[3\]](#)

1.1.2 Task 2

How a method designed for one-dimensional signals can be used to compare images or image segments (2D correlation is not in question here):

A method designed for one-dimensional signals can be adapted to compare images through several approaches:

- **Approach 1: Vectorization:** Converting the 2D image into a 1D vector by scanning row by row or column by column. This transforms the image comparison problem into a 1D signal correlation problem, allowing direct application of standard correlation formulas. [4]
- **Approach 2: Sequential Processing:** Apply 1D correlation separately to each row and then each column of the image. This approach treats the image as a collection of 1D signals and processes them independently. [5]
- **Approach 3: Template Matching via Sliding Window:** Use 1D correlation concepts within a sliding window approach. The template is moved across the image, and at each position, correlation is computed between the template and the underlying image patch by treating both as 1D vectors. [5]
- **Approach 4: Projection-Based Comparison:** Compute 1D projections of the image (e.g., row sums, column sums) and then apply 1D correlation to these projections. This captures global structural information while maintaining the 1D framework. [6]

The most practical approach for license plate character recognition would be vectorization, where each character template and image segment is flattened into a 1D vector, allowing direct application of correlation formulas for similarity measurement. That's exactly the method we will be using in the Python implementation. [7]

1.2 Image Blurring and Degradation Modeling

Blurring in images often results from motion, out-of-focus lenses, or deliberate smoothing filters. In signal processing terms, this can be modeled as the output of a system applied to an image. For example,

$$H_{\text{blur}}(z) = \frac{1 - p}{1 - pz^{-1}}$$

is a recursive 1D IIR filter (Infinite Impulse Response), often used for exponential smoothing or blurring.

1.2.1 Task 1

How the 1D filter above can be applied to images, plus the effect of the parameter p on the filter's performance:

A 1D IIR (Infinite Impulse Response) filter can be applied to 2D images through **separable processing**. The filter is applied sequentially:

1. **Row-wise processing:** Applying the 1D filter to each row of the image.
2. **Column-wise processing:** Applying the same 1D filter to each column of the resulting image.

This is mathematically equivalent to applying a 2D separable filter.

Parameter p and its effects:

The parameter p in the exponential smoothing/IIR filter $y[n] = px[n] + (1 - p)y[n - 1]$ is the **smoothing factor** or **weighting coefficient**:

- **Range:** p typically lies between 0 and 1
- **p approaching 1:** Less filtering, more responsive to input changes, preserves more high-frequency details
- **p approaching 0:** More aggressive filtering, smoother output, removes more high-frequency content
- **Effect on blurring:** Smaller p values create more pronounced blurring effects

The parameter p controls the **trade-off between responsiveness and smoothing**. In image processing, this determines how much spatial detail is preserved versus how much smoothing (blurring) is applied. **Sources used for this task:** [8] [9] [10] [11] [12]

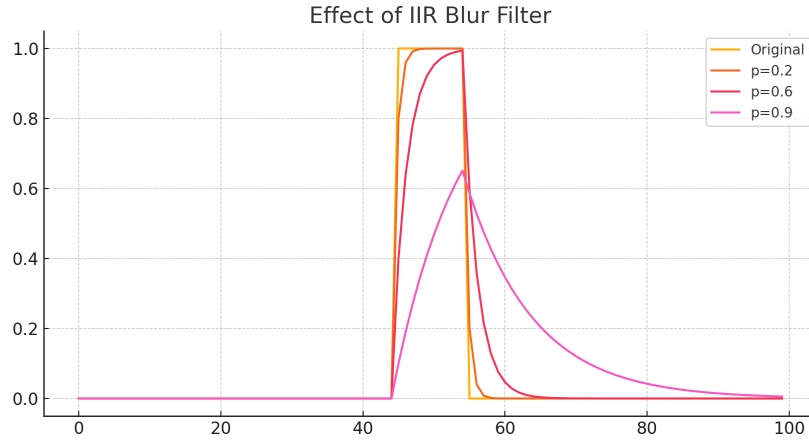


Figure 2: Effect of applying a 1D IIR blur filter with different values of p on a simple signal. Increasing p results in stronger smoothing.

1.2.2 Task 2

How we can recover a high-quality and clean image, assuming that we have the blur transformation function (blur kernel) of the channel:

Even when the blur transformation function (blur kernel) is known, perfect recovery of a high-quality, clean image is generally *not possible*. This fundamental limitation arises from several factors. First, the blurring process acts as a low-pass filter that irreversibly removes high-frequency information from the original image. Once this information is lost, it cannot be perfectly reconstructed through deconvolution alone.

Additionally, noise present in the imaging system becomes amplified during the deblurring process, particularly in frequency regions where the blur kernel has small values. This noise amplification often limits the practical effectiveness of deblurring algorithms, requiring careful regularization techniques to balance noise suppression with detail restoration.

However generally, to deblur the image and omit the effect the transfer function applied, we must apply the inverse filter to the image.

Sources used for this task: [14] [15] [16]

1.3 Sampling Rate and Resolution Effects

In image processing, sampling corresponds to the image resolution. Reducing the resolution is equivalent to down-sampling the image, which affects the frequency content and the ability to extract fine details such as the shape of characters.

In this project, we test how much the resolution can be reduced while still enabling character recognition via correlation.

1.3.1 Task

A bit of research on 2D sampling theory:

An image is considered **band-limited** if its 2D Fourier Transform is zero outside a bounded frequency region. That is:

$$X(\omega_1, \omega_2) = 0 \quad \text{for} \quad |\omega_1| > \omega_{10}, \quad |\omega_2| > \omega_{20}$$

where ω_{10} and ω_{20} represent the maximum spatial frequencies in the horizontal and vertical directions, respectively. According to the **2D Nyquist-Shannon sampling theorem**, in order to perfectly reconstruct a band-limited image without aliasing, the sampling frequencies must satisfy the following conditions:

$$f_{sx} \geq 2B_x, \quad f_{sy} \geq 2B_y$$

Here, B_x and B_y are the spatial bandwidths in the horizontal and vertical dimensions. Sampling can be modeled as multiplying the continuous image by a 2D impulse train (comb function), effectively selecting discrete grid points in space.

Aliasing and Its Consequences

If the sampling criteria are not met, **aliasing** occurs. This results in the overlapping of replicated spectra in the frequency domain, causing **high-frequency components to appear as low frequencies** in the sampled image. Aliasing distorts the true spatial structure of the image and can introduce **artifacts that are irrecoverable** during reconstruction.

Practical Implications in Character Recognition

The 2D sampling theory has significant implications for **correlation-based character recognition systems**, especially in applications like license plate recognition (LPR). Although theoretical analysis may suggest that only a few samples are needed to capture character features, real-world implementations generally require much higher sampling densities. Studies show that the most relevant spatial frequency components for character shapes lie in the range of 1 to 3 cycles per character. However, to ensure legibility and accurate recognition, it is typically necessary to have at least 8 to 12 samples across the height of each character.

This requirement stems from several practical factors, including the thickness and complexity of the font, image contrast and lighting conditions, the alignment and spacing of characters, and the specific nature of the recognition algorithm being used. In license plate recognition systems, where characters must be reliably identified under various distortions such as blur, noise, or resolution reduction, maintaining an adequate sampling density becomes essential. If the sampling rate falls below a critical threshold, key visual features may be lost—leading to blurred edges, merged fine details, and increased confusion between similar-looking characters. Therefore, sufficient resolution is not merely a theoretical preference but a practical necessity for ensuring accurate correlation-based recognition.

Sampling Density vs. Correlation Accuracy

Correlation-based recognition systems rely on matching spatial patterns between input segments and reference templates. If the sampling is too coarse:

- Distinctive features are smoothed out or aliased.
- Correlation values become unreliable.
- The system may fail to distinguish between similar characters.

Thus, for successful correlation-based matching, the sampling resolution must be sufficient to preserve the **unique structural patterns** of each character. Even if templates are accurate, **insufficient sampling can break the recognition pipeline** due to loss of discriminative information.

Conclusion

The 2D sampling theory not only underpins the theoretical foundation for digital image processing, but also plays a vital role in real-world tasks like license plate recognition. Understanding and respecting sampling constraints ensures that spatial information necessary for recognition is preserved, particularly when using correlation-based methods. Therefore, ensuring that character images are sampled at an appropriate resolution is a **non-negotiable requirement** for achieving high recognition accuracy.

Sources used for this task: [\[17\]](#) [\[18\]](#) [\[19\]](#) [\[20\]](#) [\[21\]](#) [\[22\]](#) [\[23\]](#)

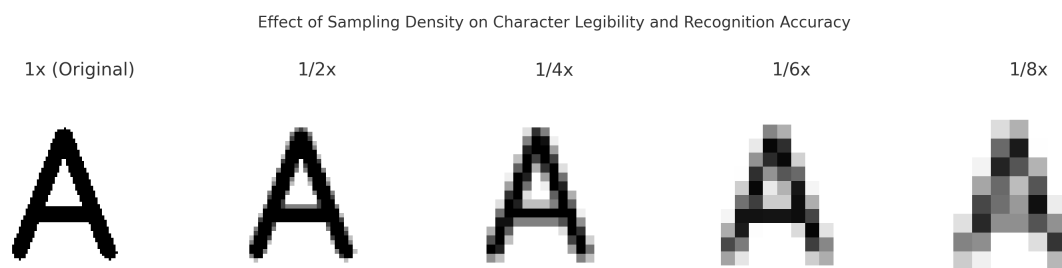


Figure 3: Effect of decreasing sampling density on character recognition. As resolution drops from left to right, the character “A” becomes increasingly distorted, illustrating how insufficient sampling can degrade legibility and impact the performance of correlation-based recognition systems.

2 Implementation

We are provided with a set of license plate images. Our task is to develop a character recognition system that can extract and identify the letters and digits from the plates. For simplicity, we assume that the system provides us with all license plates without any change in orientation or angle.

A set of images for different numbers and letters is given and we assume that characters appeared on the plates are limited to this set.

2.1 Ideal Case - Controlled Environment

In this part, we process clean and undistorted plate images where all characters are clearly visible. We assume the following simplifications:

- The resolution is sufficient for accurate character segmentation.
- There is minimal or no noise and blur.

The code referred to in this section is attached as a Jupyter Notebook file under the name 'ideal_case.ipynb'.

2.1.1 Task 1

→ *A function that takes a license plate image as input and splits the image into the individual characters and plots each.*

The function `'character_splitter'` takes a 7-element list of 4-element lists, each indicating the start and end cropping coordinates in vertical and horizontal directions of one of the seven characters in the license plate.

Due to varying character widths, we adjusted the 2D list configuration for each sample image through a minor trial and error.

Later in the `'pad_images_to_match_size'` function, we pad each of these segments with white space symmetrically, so that they become the same size with the template character images used for recognition.

2.1.2 Task 2

→ *Recognition using the algorithm of adapted 1D correlation we discussed before.*

What we basically do in the `'image_correlation_1D'` function, is that we apply the one-dimensional algorithm of correlation to the images row by row and find their correlation value. Explaining the implementation in full:

- **Converting both images to grayscale:** Image correlation works on intensity values, Thus, in general, converting to grayscale reduces each pixel to a single luminance value, simplifying computation. However, this was not necessary since the sample images in this project are not in color originally.
- **Padding images with white to make them the same size:** This is the `pad_images_to_match_size` function we mentioned before. Without padding we will have to resize the images, which may change the ratio significantly. Padding the images with white space, we attempt to minimize the reduction in accuracy.
- **Flattening to 1D and converting to Numpy array:** We first convert the 2D images into two Numpy arrays of floating point numbers. Then using the function `ravel()` we flattened it into a one-dimensional array. According to the documentation, for this function, there is an optional parameter named `'order'` having the value `'C'` by default, which means indexing the elements in row-major. [24]
- **Normalizing:** This step may not be needed at the moment because we are investigating an ideal case, but later it will be necessary for processing down sampled images. Normalizing:
 - **Makes Correlation Independent of Brightness/Contrast:** Without normalization, two images of the same digit (e.g. "5") with different brightness or contrast will have very different pixel values, even though visually they're the same.

- Ensures fair comparison in cases where the digits have noise and shadows, or the image is brighter / darker overall.
- Prevents one array from dominating: In dot-product-based correlation, unnormalized vectors let high-magnitude signals dominate the result — even if their shape doesn't match.

- **Finding the correlation value:** Finally we find the correlation value using the formula:

$$\text{corr}(f, t) = \frac{(f - \bar{f}) \cdot (t - \bar{t})}{\|f - \bar{f}\| \cdot \|t - \bar{t}\|} \quad (1)$$

We already know that perfect matches and perfect inverses have correlation values of +1 and -1 respectively, and zero means there is no correlation.

```

1 def image_correlation_1D(slice_img, template_img):
2     # Convert both to grayscale
3     slice_gray = slice_img.convert('L')
4     template_gray = template_img.convert('L')
5
6     padded_slice, padded_template = pad_images_to_match_size(slice_gray,
7         template_gray)
8
9     f = np.asarray(padded_slice, dtype=np.float32).ravel()
10    t = np.asarray(padded_template, dtype=np.float32).ravel()
11
12    # first, zero-centering:
13    f_mean, t_mean = f.mean(), t.mean()
14    f_centered = f - f_mean
15    t_centered = t - t_mean
16
17    # Compute denominator (avoiding zero division)
18    f_norm = np.linalg.norm(f_centered)
19    t_norm = np.linalg.norm(t_centered)
20    eps = 1e-8
21    if f_norm < eps or t_norm < eps:
22        return -1.0
23    # Correlation Coefficient
24    correlation_coeff = (f_centered @ t_centered) / (f_norm * t_norm)
25    return correlation_coeff

```

We have another function named 'LPR' which is in charge of comparing each of the 7 characters in the license plate (outputs of the segmentation function) to the template characters (by calling the 1D correlation function) and finding out for which of them the correlation value is the maximum. That's how we recognize the whole license plate number.

2.1.3 Task 3

→ *Applying down sampling of different rates to the images and see until when the recognition is still accurate and reliable.*

The 'down_sampler' function we have written, gets a down sampling factor as an input and selects some pixels of the image based on its value and throws the rest away, introducing intentional blockiness and inaccuracy to the license plate image. For example, a factor of 0.33 means that we approximately choose 1 out of each 3 pixels.

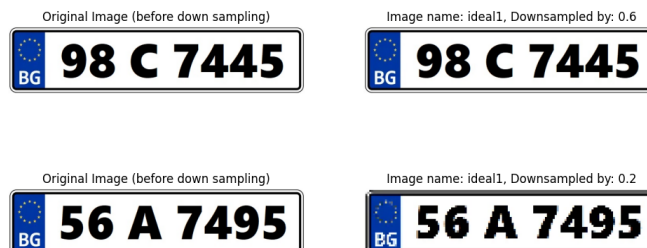
At the end we resize the degraded image back to the initial dimensions, so that comparing it with the original version would be easier. To do so, we set the optional parameter of the `resize()` function, named 'resample' to `PIL.Image.NEAREST`. This prevents automatic interpolation and preserves the blockiness in the resized image. (For example, one of the other options we have for the value of 'resample' is `PIL.Image.BILINEAR` that does linear interpolation. **References for this part:** [25], [26])

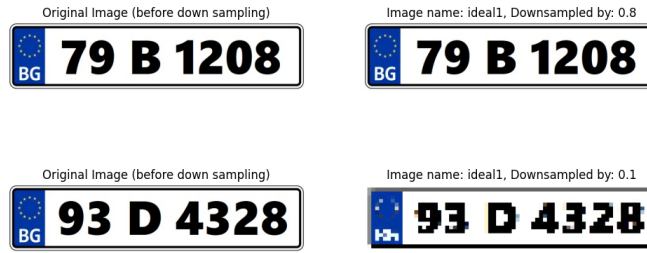
```

1 # PART 1: Function for downsampling:
2
3 def down_sampler(input_path, image_name, output_dir, factor,
4                 result_show=False, save_result=False):
5     img = Image.open(input_path)
6     original_size = img.size # (width height)
7
8     # converting image to numpy array
9     img_np = np.array(img)
10
11     # First checking if the down sampling factor is within the valid interval
12     if $factor \leq 0$ or factor > 1:
13         raise ValueError("Downsampling factor not within the valid region.")
14
15     # Now we determine the step size based on the down sampling factor
16     step = int(1 / factor)
17
18     # Now extracting the subset: taking every 'step'th pixel row-wise and
19     # column-wise:
20     img_small = img_np[::step, ::step]
21
22     # converting the down sampled image back to a PIL image:
23     img_small_pil = Image.fromarray(img_small)
24
25     # Now resizing the degraded image back to the original dimensions
26     img_degraded = img_small_pil.resize(original_size, Image.NEAREST)
27
28     if result_show:
29         plt.figure(figsize=(12, 5))
30         plt.subplot(1, 2, 1)
31         plt.title("Original Image (before down sampling)")
32         plt.imshow(img, cmap='gray')
33         plt.axis('off')
34
35         plt.subplot(1, 2, 2)
36         plt.title(f"Image name: {image_name}, Downsampled by: {factor}")
37         plt.imshow(img_degraded, cmap='gray')
38         plt.axis('off')
39         plt.show()
40
41     output_filename = None
42     if save_result:
43         if not os.path.exists(output_dir):
44             os.makedirs(output_dir)
45         output_filename = f"{image_name}_dwnsmpl_{factor:.2f}"
46         img_degraded.save(str(output_dir) + f"/{output_filename}.png")
47
48
49     return img_degraded, output_filename

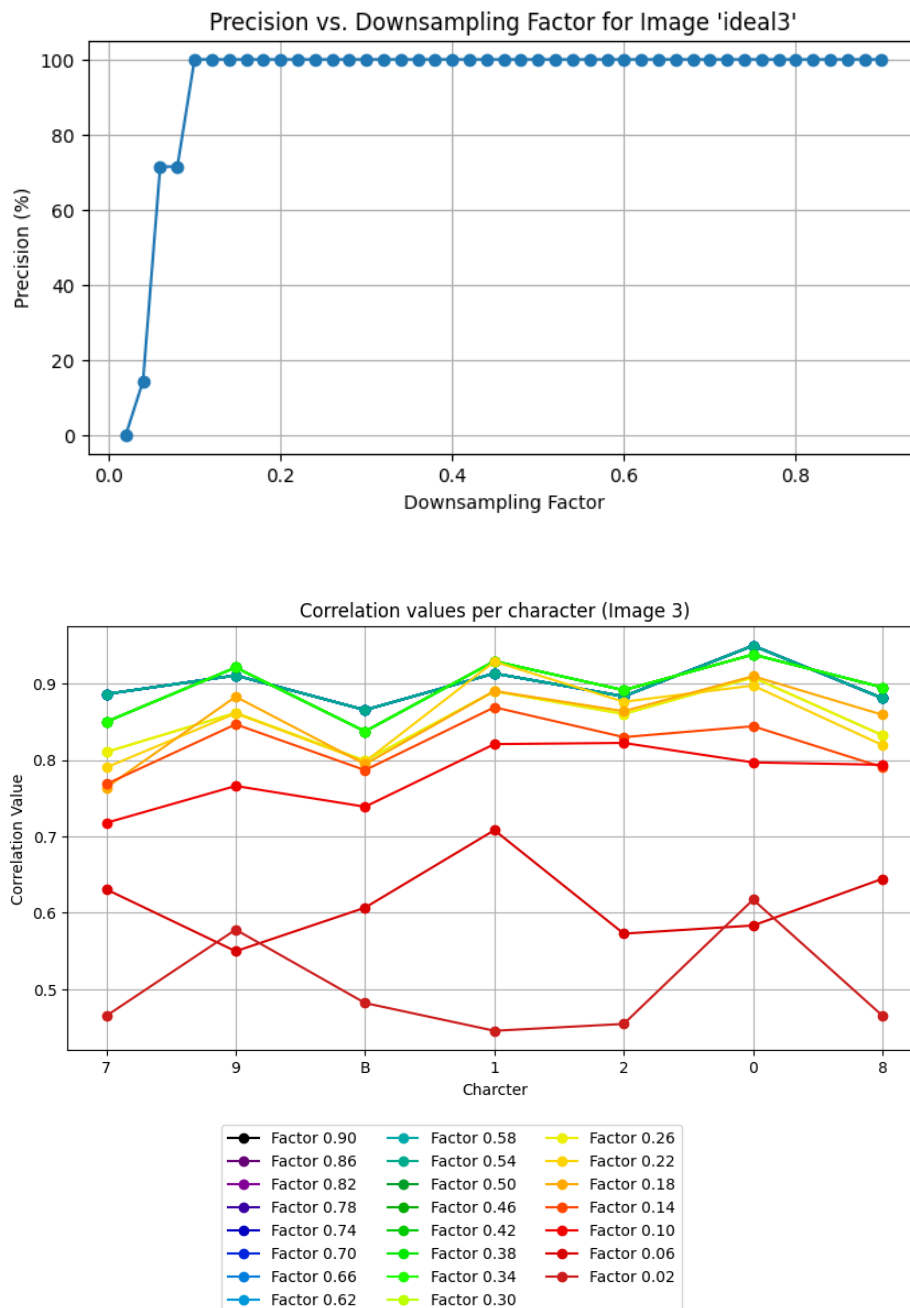
```

Here are some examples of how the function affects the image:





Now we pass each image to the segmentation function and while test the down sampling function on it for different rates, we track how the overall accuracy percentage and correlation value per character change.
As an example, these are the results of processing the third sample image this way:



In the latter plot, the reason why some traces in the correlation values per character are not shown is that the correlation values for larger down sampling rates (approx. 0.34 and above) are so close together that they are regarded as the same values, resulting in fully overlapping traces. A common practice in all four result sets is that correlation values change much slower for greater sampling rates, whereas for rates below 0.14, changes in accuracy and correlation values are much more drastic.

The minimum acceptable downsampling rate is defined as the threshold value above which full accuracy (100%) is preserved for all higher sampling rates. We found this value to be 0.07, 0.1, 0.1 and 0.1 for the given samples from the first to the fourth, respectively,

2.2 Realistic Case – Simulated Real-World Conditions

In this part, we will work with blurred versions of the same license plates. These images simulate realistic degradation caused by optical imperfections or motion blur. However, we are not given the exact blur kernel. Our goal is to analyze and reverse the degradation effects as far as possible.

The code referred to in this section is attached as a Jupyter Notebook file under the name 'real_case.ipynb'.

2.2.1 Task 1

→ *Testing whether character recognition is still successful with the previous methods and without any deblurring.*
 Passing the blurred images to the segmentation and then to the LPR functions, we realize that the answer is negative and it's not possible to recognize the plates accurately without deblurring. For example, '98 C 7445' is recognized as 'B B A B D A B' and '56 A 7495' as 'A A 311 A A' which are so off the track.

2.2.2 Task 2

→ *Estimating the optimal value for p for deblurring.*

The structure of the transfer function applied to the images for blurring them is given as:

$$H_{blur}(z) = \frac{1-p}{1-pz^{-1}} \quad (2)$$

where we do not know about the value of p and intend to find it such that the resulting deblurred images are the most clear. $H_{blur}(z)$ is:

- Low-pass: By replacing z with $e^{j\omega}$, we realize that for $0 \leq p < 1$, the gain is max at $\omega = 0$ and then decreases as ω approaches π .
- IIR: There is z^{-1} in the denominator.
- Causal (the ROC including $|z| = 1$) and stable ($|z| < 1$ for all poles) only if $0 \leq p < 1$.

The function 'deblur' is in charge of applying the inverse of this transfer function, $H_{blur}^{-1}(z) = \frac{1-pz^{-1}}{1-p}$ and an FIR filter.

Algebraically, any value for p is allowed except for 1. However, exceeding too much from 1 may drastically amplify the noise and result in a distorted image or at least an indifferent one, rather than a clearer version. So although $p \neq 1$ is allowed, we don't expect the optimal p to be far from 1.

Also, it's possible to apply the inverse filter vertically (column by column) or horizontally (row by row). For the trial and errors below, we've used the first sample image and assumed that the application is done horizontally.



2.2.3 Task 3

→ *Deblurring the sample images with the best configuration (value for p + direction of applying).*

According to the trial and error we did, a good choice may be $p = 0.994$ (for p greater than 1 there is approximately no visible change). This configuration works well for sample images 1, 3 and 4:



(a) Sample Image No. 1



(b) Sample Image No. 2



(c) Sample Image No. 3

However, it doesn't work properly for the second sample:



You can visually say that the noise on this image is vertical compared to the others, so we tried vertical application of the inverse filter and also modifying the value p a bit, we choose it to be 0.984:



2.2.4 Task 4

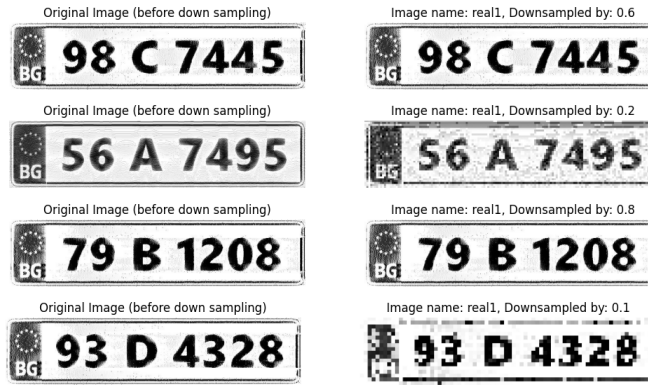
→ *Passing the deblurred images to the character recognition function and seeing if the results are accurate.*

Turns out with the previous segmentation coordinates and the configurations we just chose the LPR function is working fine; all the recognition results are accurate and correlation values are pretty high. Results are available in the Jupyter notebook.

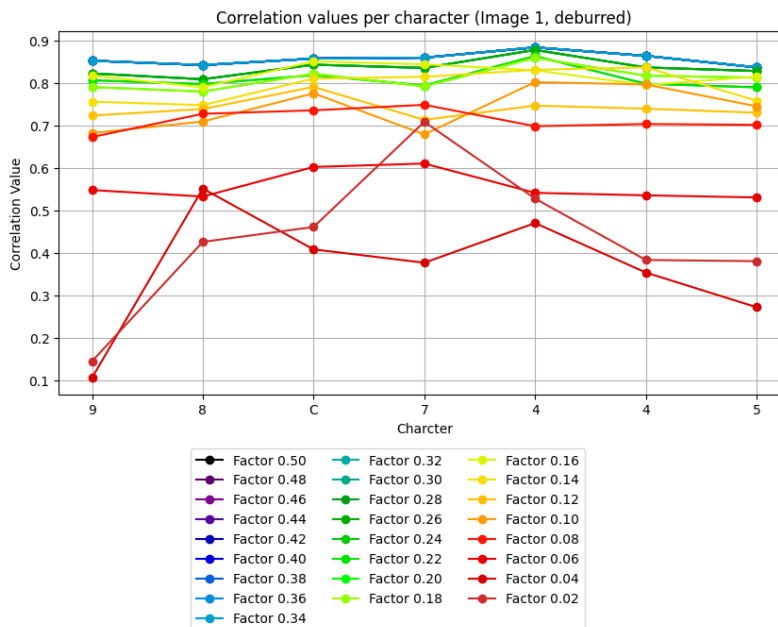
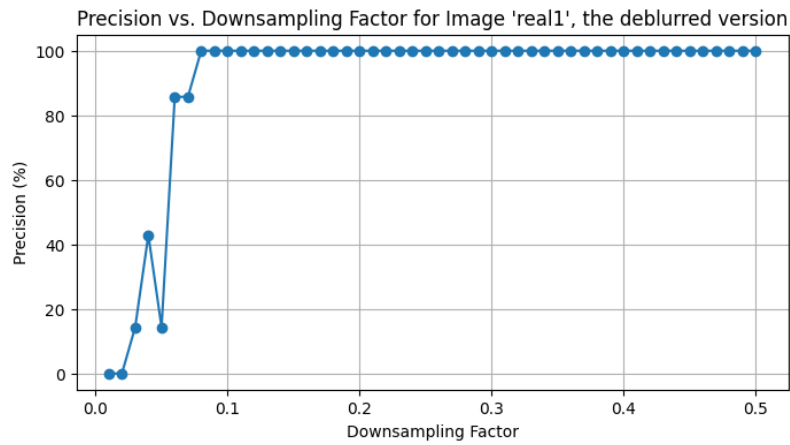
2.2.5 Task 5

→ *Applying down sampling of different rates to the deblurred images and see until when the recognition is still accurate and reliable.*

Some examples of the deblurred images after down sampling:



Now iterating through different down sampling rates and tracking the changes the same way we did for the ideal case, here are the outputs of processing the first image as an example:



Again some traces are overlapping in the latter plot. We find the minimum acceptable sampling rate (definition the same as before) to be 0.08

2.2.6 Task 6

→ *Determining which one of the two given noisy images has higher-frequency noise.*

The structure of the reverse transfer function $H_{blur}^{-1}(z)$ used in the deblurring function is $\frac{1-pe^{j\omega}}{1-p}$. As w increases, its magnitude increases. Thus, it's high pass and suppresses lower frequencies more.

So, we can apply the same deblurring function we had to both the images. The image with higher frequency noise, will become more distorted and noisier.

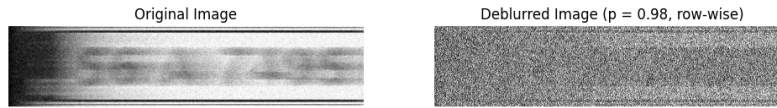


Figure 7: The first given noisy image after applying 'deblur'

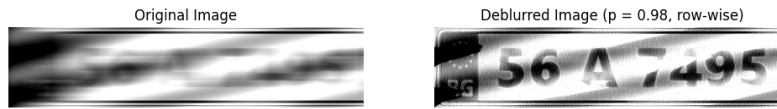


Figure 8: The second given noisy image after applying 'deblur'

We shall conclude that the first image has higher frequency noise,